Programação Estruturada

Estrutura Condicional

Exercícios – Sem Condicional

1. Faça um programa que leia 3 notas e escreva a média aritmética
2. Faça um programa que leia a base e altura de um triângulo e que escreva a sua área

Estrutura Condicional

- IF – ELSE
- O comando IF – ELSE é usado para expressar decisões, sua sintaxe é:

```
if (expressão) {
        comandoA1;
        comandoA2;
} else {
        comandoB1;
        comandoB2;
        comandoB3;
}
```

Se (expressão) for verdadeira executa-se a lista de *comandosA*. Caso contrário, executa-se a lista de *comandosB*.

Estrutura Condicional

- Os comandos de cada bloco são delimitados por { e }
- Caso só exista um comando no bloco, então as chaves podem ser suprimidas

```
if (expressão)
        comandoA;
else
        comandoB;
```

- A parte do `else` é opcional

Estrutura Condicional

- Sendo o `else` opcional, pode existir “dúvida” quando ele é omitido em um encadeamento de ifs
- Exemplo:

```
if (n > 0)
    if (a > b)
        z = a;
    else
        z = b;
```

- A ambiguidade é resolvida associando o `else` ao `if` mais interno.

Estrutura Condicional

- Caso o programador deseje associar o `else` ao `if` mais externo, é necessário o uso de chaves.
- Exemplo:

```
if (n > 0) {
    if (a > b)
        z = a;
} else
        z = b;
```

Encadeamento de IFs

- Exemplo:

```
if (expressão1)
        comando1;
else
        if (expressão2)
            comando2;
        else
            if (expressão3)
                comando3;
            else
                if (expressão4)
                    comando4;
                else
                    comando5;
```

Exercícios

1. Faça um programa que leia 3 valores inteiros (diferentes) e escreva qual o maior.
2. Faça um programa que leia 3 valores reais e que verifique se eles formam um triângulo. Em caso positivo, indique se o triângulo é equilátero, isósceles ou escaleno.

SWITCH

- O comando `switch` é uma estrutura de decisão múltipla que testa se uma expressão combina com um de vários valores constantes
- O fluxo de controle é desviado para a instrução apropriada de acordo com o resultado da comparação.

Switch

```
switch (expressão) {
        case  exp_constanteA:
            comandoA1;
            comandoA2;
            ...
            break;
        case  exp_constanteB:
            comandoB1;
            comandoB2;
            ...
            break;
        default:
            comandoX1;
            ...
            break;
}
```

Switch

- O comando `break` é importante para finalizar cada caso do switch
- A cláusula `default` é opcional. Ela é executada apenas quando nenhum dos casos combinar com a expressão
- No case, não é necessário usar chaves, mesmo que tenhamos mais de um comando → o `break` indicará o final de cada caso
- Se não for usado o `break` para finalizar um caso a execução prossegue para o próximo → Atenção!!

Switch - Exemplo

```
int diaSemana;

...

switch (diaSemana) {
        case  1:
            printf("Domingo");
            break;
        case  2:
            printf("Segunda");
            break;
        case  3:
            printf("Terça");
            break;
        case  4:
            printf("Quarta");
            break;
        case  5:
            printf("Quinta");
            break;
        case  6:
            printf("Sexta");
            break;
        case  7:
            printf("Sábado");
            break;
        default:
            printf("Mês Inválido");
            break;
}
```

Exercício

- Faça um programa em C que leia uma data no formato "dd/mm/aaaa" e que escreva a mesma data por extenso. Utilize o comando `switch case`.

- Exemplo:

  Entrada: 10/12/2021

  Saída: 10 de dezembro de 2021

Operador Condicional

- Exemplo:

```
if (exp1)
        exp2;
else
        exp3;
```

- Equivale a:

```
(exp1) ?  exp2: exp3;
```

Operador Condicional

- Exemplo:

```
if (a > b)
        z = a;
else
        z = b;
```

- Equivale a:

```
z = (a > b)  ?  a : b;
```

Operador Condicional

- Exemplo:

```
if (x > 0)
        x++;
else
        x--;
```

- Equivale a:

```
(x > 0)  ?  x++ : x--;
```